Job Brief: Systems Programmer for TSC-1000 Controller Firmware Enhancement

Introduction: We are seeking an experienced embedded systems programmer to enhance the firmware of the TSC-1000 controller. This controller comprises two boards: an ESP32 microcontroller board responsible for the core control logic, and a Texas Instruments-based power management board that handles motor power delivery. The system is designed to manage electric motors using a 24V DC input to provide a 3-phase AC output, controlling a four-head compressor for various applications at a rate of 1.1 Standard Cubic Feet per Minute (SCFM), typically operating within the 20-40 PSI range.

Motor Models and Electrical Specifications:

The TSC-1000 controls four distinct motor models based on maximum current draw. This will impact the overcurrent protection (OCP) logic to be implemented in the firmware:

Motor Model	Maximum Current (Amps)
TL20	6 A
TL35	8 A
TL50	10 A
TL70	12 A

The ESP32 currently operates with firmware that provides basic control and operational stability. However, key enhancements and fixes are needed to improve overall performance in the field.

Task Requirements:

1. **Immediate High Amp Shutoff Enhancement**
    - **Current Situation**: Present firmware incorporates a delayed high amp shutoff mechanism that triggers if the motor current exceeds 1.5 times the configured limit for 15 seconds, with a warning if it is 1.2 times for 2 minutes.
    - **Technical Task:** Modify the Overcurrent Protection (OCP) algorithm in the ESP32 firmware to incorporate an immediate motor shutdown when the current draw reaches the set OCP DC Amp value. This should involve revising the firmware logic to:
        - Implement an interrupt-driven process to trigger immediate shutoff when the threshold is reached.
        - Ensure compatibility with all motor models by referencing the current draw limits from a lookup table.
2. **Cumulative Runtime Counter Adjustment**
    - **Current Situation:** The system utilizes a counter for runtime, which currently resets to zero upon reaching approximately 96 hours (4 days).
    - **Technical Task A:** Modify the firmware to allow for indefinite tracking of runtime using a persistent memory storage to ensure the runtime is retained through power cycles.
    - **Technical Task B:** Adjust the LCD display logic to initially show runtime in minutes and dynamically convert to hours.
3. **AutoStart on Voltage Recovery**
    - **Current Situation:** The system does not automatically restart when input voltage drops and then returns to a steady state.

- **Technical Task:** Implement a voltage monitoring routine using the ESP32's ADC to detect drops in the input voltage. Integrate a mechanism to ensure the system reliably restarts when voltage stabilizes above the defined threshold. This should include:
  - Configuring to filter transient voltage spikes.
  - Updating the firmware to trigger a system reboot upon voltage recovery.

4. **Verification and Correction of Error Codes**
   - **Technical Task A:** Review the error-handling in the firmware to verify the accuracy of error codes displayed. Adjust error reporting mechanisms, ensuring:
     - Accurate status code mapping for each operational fault detected.
     - Proper triggering and resetting of error flags in the system registers.
   - **Technical Task B:** Document each error code within the firmware's Error Code Library to standardize the descriptions for future diagnostics.
5. **Sensor Error Code Review**
   - **Current Situation:** The system triggers a sensor error code approximately every 3 months due to a suspected code logic issue.
   - **Technical Task:** Conduct a code review of the sensor monitoring, focused on preventing the intermittent triggering of the sensor fault.
6. **Communication and Remote Control Configuration**
   - **Current Situation:** The controller is equipped with an RS485 interface for remote control and has four programmable digital inputs (DIs) and digital outputs (DOs) that are currently unconfigured.
   - **Technical Task:** Develop a Modbus RTU communication protocol implementation over the RS485 interface to enable remote access. Additionally:
     - Configure the General-Purpose Input/Output (GPIO) pins for the DIs and DOs on the ESP32, allowing conditional logic control.
     - Create a command-set library to handle remote operations and feedback, ensuring it can be integrated with SCADA systems or other remote monitoring solutions.
     - Provide us with the final registry mapping.

Candidate Requirements:

- Extensive experience with embedded firmware development, particularly with ESP32/Arduino.
- Proficiency in implementing communication protocols (e.g., Modbus RTU) over RS485 and configuring GPIO for digital control.
- Deep understanding of motor control firmware, power electronics, and overcurrent protection mechanisms.
- Strong debugging skills, including use of tools such as JTAG debuggers and logic analyzers.

Deliverables:

- Enhanced firmware with the aforementioned functionality implemented and tested.
- A comprehensive technical documentation update, including new error code definitions and a communication interface manual.
- Test reports demonstrating the stability and accuracy of the firmware under various operational scenarios.